Relatório Técnico: Heurística Construtiva e Meta-heurística GRASP para o Caminho Hamiltoniano Mínimo

Tailan de Souza Oliveira Rubens Rabelo Soares
Curso de Sistemas de Informação – Universidade Federal do Ceará (Campus Quixadá)
Disciplina: Pesquisa Operacional – Prof. Dr. Paulo Henrique Macêdo

Outubro de 2026

1. Metodologia e Funcionamento dos Algoritmos

Este trabalho resolve o Problema do Caminho Hamiltoniano Mínimo (PCHM) em grafos direcionados com origem fixa no vértice $s = 1$ e destino fixo no vértice $t = n$. Os testes foram realizados em Julia 1.12.6 em um computador Linux x86_64 (Intel Core i5-12450HX com 12 threads e 15,34 GB de RAM DDR5), usando semente fixa (`seed = 42`) para garantir que os resultados possam ser reproduzidos.

Na Atividade 1, foi implementada a heurística construtiva inicial baseada na regra do vizinho mais próximo. O algoritmo começa no vértice 1 e, a cada passo, escolhe o vértice intermediário ainda não visitado que possui a viagem mais barata a partir do ponto atual. Quando todos os nós intermediários já foram visitados, o algoritmo liga o último vértice diretamente ao destino final n. Por olhar apenas para o próximo passo imediato, essa abordagem é muito rápida, mas costuma deixar conexões caras para o final do caminho.

Na Atividade 2, foi implementada a meta-heurística GRASP com 1000 iterações e fator de aleatoriedade $\alpha = 0,15$. Em cada iteração, o algoritmo primeiro constrói um caminho escolhendo arestas de forma sorteada dentro de uma Lista Restrita de Candidatos (LRC), que reúne apenas as arestas mais baratas que não violam os graus dos nós nem formam ciclos antes da hora. Para verificar se uma nova aresta fecharia um ciclo indevido, testamos duas formas: uma usando a estrutura de conjuntos disjuntos (*Union-Find*) e outra percorrendo os nós diretamente. Depois de construir o caminho, o algoritmo aplica a busca local VND (*Variable Neighborhood Descent*), que tenta melhorar a rota testando três tipos de modificações nos nós intermediários: o 2-Opt (que inverte um trecho da rota), o Or-Opt (que move blocos de 1, 2 ou 3 nós seguidos para outra posição) e o Swap (que troca dois nós de lugar). Para ganhar velocidade, a busca local calcula apenas a diferença de custo das arestas que mudaram de posição, sem precisar somar a rota inteira novamente.

2. Resultados Computacionais e Comparação com o Modelo PLI

A Tabela 1 reúne os custos e os tempos de execução das 13 instâncias da TSPLIB obtidos pela Heurística Inicial, pelo GRASP (com e sem *Union-Find*) e pelo modelo exato de Programação Linear Inteira (PLI com Gurobi do Trabalho Prático 01, limitado a 180s). A coluna Melhoria mostra quanto o GRASP reduziu o custo em relação à Heurística Inicial ($\frac{Z_{\text{Inicial}} - Z_{\text{GRASP}}}{Z_{\text{Inicial}}} \times 100\%$), enquanto a coluna Gap_{PLI} mostra a diferença percentual entre o GRASP e o resultado do modelo exato ($\frac{Z_{\text{GRASP}} - Z_{\text{PLI}}}{Z_{\text{PLI}}} \times 100\%$).

Tabela 1: Comparação entre Heurística Inicial, Meta-heurística GRASP (1000 iterações, $\alpha = 0,15$) e Modelo Exato PLI (TP 01).

Instância	n	Z_{Inicial}	T_{Inicial} (s)	Z_{GRASP}	$T_{\text{GRASP}}^{\text{Com UF}}$ (s)	$T_{\text{GRASP}}^{\text{Sem UF}}$ (s)	Melhoria (%)	Z_{PLI} (TP1)	T_{PLI} (s)	Gap_{PLI} (%)
`br17.atsp`	17	56,00	$< 0,0001$	34,00	0,2711	0,2296	39,29%	34,00	0,32	0,00%
`ft53.atsp`	53	9583,00	$< 0,0001$	6950,00	10,3490	11,8697	27,48%	6196,00	12,37	12,17%
`ft70.atsp`	70	42057,00	$< 0,0001$	39608,00	30,6039	34,9154	5,82%	37655,00	2,06	5,19%
`ftv33.atsp`	34	1547,00	$< 0,0001$	1275,00	1,3121	1,3420	17,58%	1275,00	0,47	0,00%
`ftv35.atsp`	36	1750,00	$< 0,0001$	1455,00	1,5804	1,7806	16,86%	1443,00	0,35	0,83%
`ftv38.atsp`	39	1828,00	$< 0,0001$	1520,00	2,2062	2,4271	16,85%	1511,00	0,39	0,60%
`ftv44.atsp`	45	1933,00	$< 0,0001$	1654,00	3,8957	4,4952	14,43%	1614,00	0,95	2,48%
`ftv47.atsp`	48	2298,00	$< 0,0001$	1796,00	5,5489	5,8328	21,85%	1754,00	1,43	2,39%
`ftv55.atsp`	56	2052,00	$< 0,0001$	1696,00	9,3377	10,1509	17,35%	1613,00	2,06	5,15%
`ftv64.atsp`	65	2586,00	$< 0,0001$	1975,00	24,8442	27,1173	23,63%	1840,00	6,11	7,34%
`ftv70.atsp`	71	2637,00	$< 0,0001$	2104,00	34,0494	37,3969	20,21%	1958,00	6,89	7,46%
`p43.atsp`	43	642,00	$< 0,0001$	596,00	5,6638	5,5128	7,17%	589,00*	180,04	1,19%
`ry48p.atsp`	48	17971,00	$< 0,0001$	14231,00	7,9283	7,7672	20,81%	14070,00	1,70	1,14%

*Em `p43.atsp`, o modelo PLI atingiu o tempo limite de 180s sem provar otimalidade global, retornando a melhor solução encontrada (589, 00).

3. Análise dos Resultados e Principais Diferenças

Na comparação entre a Heurística Inicial e o GRASP, a principal diferença aparece na qualidade das rotas geradas. Embora a heurística inicial termine quase instantaneamente (menos de 0, 0001 s em todos os testes), ela toma decisões ruins no final do percurso por escolher sempre a aresta mais barata do momento sem planejar a chegada ao destino n. O GRASP conseguiu melhorar o resultado inicial em todas as 13 instâncias, com uma redução média

de custo de 19, 18%. As maiores diferenças de custo entre os dois métodos ocorreram em `br17.atsp`, onde o custo caiu de 56, 00 para 34, 00 (economia de 39, 29%), em `ft53.atsp`, onde caiu de 9583, 00 para 6950, 00 (economia de 27, 48%), e em `ftv64.atsp`, com redução de 23, 63%. Mesmo nas instâncias onde a heurística inicial já partia de um valor mais próximo, como `ft70.atsp` e `p43.atsp`, o GRASP ainda reduziu o custo em 5, 82% e 7, 17%, respectivamente.

Quando comparamos o GRASP com o modelo exato de Programação Linear Inteira (PLI) do Trabalho Prático 01, observamos que o GRASP encontrou exatamente a solução ótima ($\text{Gap}_{\text{PLI}} = 0, 00\%$) em `br17.atsp` (34, 00) e `ftv33.atsp` (1275, 00), além de ficar muito próximo do ótimo (diferença abaixo de 2, 50%) em outras seis instâncias: `ftv38.atsp` (0, 60%), `ftv35.atsp` (0, 83%), `ry48p.atsp` (1, 14%), `p43.atsp` (1, 19%), `ftv47.atsp` (2, 39%) e `ftv44.atsp` (2, 48%). A maior diferença de qualidade a favor do PLI ocorreu em `ft53.atsp`, na qual o GRASP ficou 12, 17% acima do ótimo exato (6950, 00 contra 6196, 00), seguida pelas instâncias maiores `ftv70.atsp` (7, 46%) e `ftv64.atsp` (7, 34%), onde o espaço de busca cresce muito para 1000 iterações.

Em relação ao tempo de execução entre GRASP e PLI, a maior diferença a favor do GRASP aconteceu na instância `p43.atsp`: enquanto o solver exato Gurobi travou na árvore de busca e atingiu o tempo máximo de 180, 04 s, o GRASP entregou uma solução apenas 1, 19% acima em 5, 66 s, sendo mais de 31 vezes mais rápido. Na instância `ft53.atsp`, o GRASP também terminou em tempo menor que o PLI (10, 35 s contra 12, 37 s). Por outro lado, a maior diferença de tempo a favor do PLI ocorreu nas instâncias com mais de 60 vértices, especialmente em `ft70.atsp` e `ftv70.atsp`, nas quais o Gurobi levou 2, 06 s e 6, 89 s, enquanto o GRASP precisou de 30, 60 s e 34, 05 s porque a busca local Or-Opt testa combinações na ordem de $O(n^3)$ em cada uma das 1000 iterações.

Por fim, comparando as duas versões do GRASP (com e sem *Union-Find*), ambas chegaram exatamente aos mesmos custos finais por usarem a mesma semente aleatória, mas o tempo de execução mostrou diferenças importantes conforme o grafo cresce. Nas instâncias pequenas ou médias como `br17.atsp` e `p43.atsp`, a versão sem *Union-Find* teve tempo praticamente igual ou frações de segundo menor devido à simplicidade de percorrer poucos nós. Já nas maiores instâncias, o *Union-Find* fez diferença clara no tempo total, economizando 4, 31 s em `ft70.atsp` (12, 3% mais rápido), 3, 35 s em `ftv70.atsp` (9, 0% mais rápido), 2, 27 s em `ftv64.atsp` (8, 4% mais rápido) e 1, 52 s em `ft53.atsp` (12, 8% mais rápido).

4. Rotas Encontradas pela Meta-heurística GRASP

br17 (34,00): 1 → 12 → 3 → 14 → 2 → 13 → 11 → 10 → 15 → 7 → 6 → 16 → 5 → 4 → 8 → 9 → 17.
ft53 (6950,00): 1 → 37 → 33 → 34 → 27 → 26 → 28 → 30 → 29 → 32 → 31 → 5 → 3 → 16 → 38 → 36 → 41 → 40 → 39 → 46 → 43 → 47 → 44 → 42 → 48 → 45 → 35 → 19 → 18 → 17 → 24 → 21 → 22 → 25 → 23 → 20 → 2 → 4 → 12 → 11 → 13 → 15 → 14 → 10 → 9 → 8 → 7 → 6 → 52 → 51 → 49 → 50 → 53.
ft70 (39608,00): 1 → 2 → 61 → 60 → 11 → 15 → 64 → 67 → 65 → 69 → 66 → 68 → 36 → 39 → 14 → 9 → 47 → 46 → 63 → 62 → 59 → 58 → 45 → 49 → 44 → 50 → 48 → 56 → 54 → 24 → 22 → 27 → 3 → 5 → 28 → 23 → 26 → 4 → 6 → 7 → 17 → 16 → 29 → 55 → 53 → 52 → 51 → 19 → 20 → 31 → 35 → 33 → 30 → 13 → 10 → 12 → 8 → 21 → 18 → 25 → 57 → 34 → 32 → 37 → 43 → 42 → 41 → 40 → 38 → 70.
ftv33 (1275,00): 1 → 2 → 3 → 4 → 14 → 13 → 15 → 16 → 17 → 26 → 25 → 24 → 27 → 28 → 29 → 30 → 23 → 21 → 22 → 32 → 19 → 20 → 18 → 12 → 9 → 11 → 10 → 33 → 8 → 5 → 7 → 6 → 31 → 34.
ftv35 (1455,00): 1 → 14 → 13 → 6 → 7 → 8 → 35 → 9 → 10 → 12 → 15 → 16 → 17 → 27 → 2 → 4 → 3 → 5 → 33 → 31 → 28 → 24 → 26 → 25 → 18 → 11 → 34 → 19 → 20 → 21 → 22 → 23 → 29 → 30 → 32 → 36.
ftv38 (1520,00): 1 → 17 → 16 → 9 → 38 → 10 → 11 → 13 → 12 → 15 → 18 → 19 → 20 → 2 → 4 → 3 → 5 → 6 → 8 → 7 → 36 → 34 → 30 → 29 → 28 → 21 → 14 → 37 → 22 → 23 → 24 → 25 → 26 → 27 → 31 → 32 → 33 → 35 → 39.
ftv44 (1654,00): 1 → 22 → 20 → 15 → 11 → 44 → 12 → 13 → 14 → 16 → 17 → 18 → 19 → 43 → 23 → 24 → 25 → 26 → 27 → 28 → 29 → 31 → 30 → 33 → 34 → 35 → 37 → 38 → 4 → 5 → 6 → 21 → 8 → 10 → 9 → 41 → 42 → 7 → 40 → 39 → 36 → 32 → 2 → 3 → 45.
ftv47 (1796,00): 1 → 26 → 2 → 10 → 34 → 28 → 4 → 25 → 5 → 30 → 31 → 6 → 32 → 7 → 11 → 9 → 12 → 38 → 19 → 18 → 13 → 33 → 8 → 24 → 35 → 14 → 47 → 37 → 15 → 36 → 16 → 17 → 46 → 21 → 39 → 27 → 43 → 29 → 3 → 42 → 44 → 23 → 20 → 45 → 40 → 22 → 41 → 48.
ftv55 (1696,00): 1 → 34 → 2 → 4 → 49 → 32 → 47 → 35 → 3 → 15 → 11 → 8 → 33 → 9 → 37 → 10 → 38 → 12 → 52 → 13 → 16 → 17 → 18 → 53 → 27 → 26 → 25 → 43 → 22 → 23 → 42 → 51 → 24 → 55 → 28 → 50 → 30 → 21 → 41 → 20 → 19 → 40 → 39 → 14 → 36 → 5 → 7 → 6 → 48 → 45 → 29 → 54 → 44 → 46 → 31 → 56.
ftv64 (1975,00): 1 → 16 → 62 → 51 → 29 → 28 → 27 → 23 → 24 → 31 → 54 → 32 → 63 → 53 → 52 → 25 → 26 → 64 → 30 → 33 → 55 → 2 → 38 → 4 → 19 → 18 → 17 → 14 → 47 → 46 → 11 → 61 → 15 → 48 → 8 → 37 → 9 → 41 → 42 → 10 → 43 → 44 → 45 → 12 → 13 → 36 → 20 → 21 → 22 → 49 → 50 → 60 → 56 → 34 → 57 → 39 → 3 → 5 → 40 → 6 → 7 → 58 → 59 → 35 → 65.
ftv70 (2104,00): 1 → 29 → 26 → 28 → 7 → 14 → 67 → 15 → 13 → 16 → 17 → 19 → 20 → 21 → 22 → 23 → 24 → 61 → 33 → 34 → 70 → 42 → 64 → 41 → 69 → 47 → 63 → 53 → 46 → 54 → 45 → 44 → 43 → 36 → 35 → 37 → 38 → 39 → 40 → 48 → 49 → 52 → 51 → 68 → 50 → 55 → 56 → 4 → 5 → 9 → 8 → 10 → 12 → 11 → 59 → 62 → 60 → 58 → 57 → 6 → 32 → 31 → 30 → 25 → 27 → 18 → 65 → 66 → 2 → 3 → 71.
p43 (596,00): 1 → 5 → 38 → 7 → 6 → 8 → 12 → 10 → 11 → 9 → 33 → 32 → 30 → 31 → 35 → 34 → 15 → 13 → 14 → 16 → 18 → 17 → 21 → 19 → 20 → 27 → 26 → 25 → 22 → 24 → 23 → 36 → 37 → 3 → 2 → 4 → 29 → 28 → 40 → 39 → 42 → 41 → 43.

ry48p (14231,00): $1 \to 9 \to 40 \to 15 \to 12 \to 47 \to 20 \to 33 \to 46 \to 36 \to 7 \to 18 \to 28 \to 30 \to 37 \to 43 \to 17 \to 27 \to 19 \to 6 \to 44 \to 31 \to 38 \to 8 \to 16 \to 22 \to 3 \to 41 \to 34 \to 11 \to 23 \to 14 \to 25 \to 13 \to 21 \to 39 \to 32 \to 24 \to 10 \to 45 \to 35 \to 4 \to 26 \to 42 \to 2 \to 29 \to 5 \to 48.$

5. Representação Gráfica das Rotas (GRASP)

As Figuras 1 a 13 exibem o mapeamento bidimensional das rotas geradas pelo GRASP nas 13 instâncias da TSPLIB, destacando a transição contínua da origem ($s = 1$, verde) ao destino ($t = n$, vermelho).

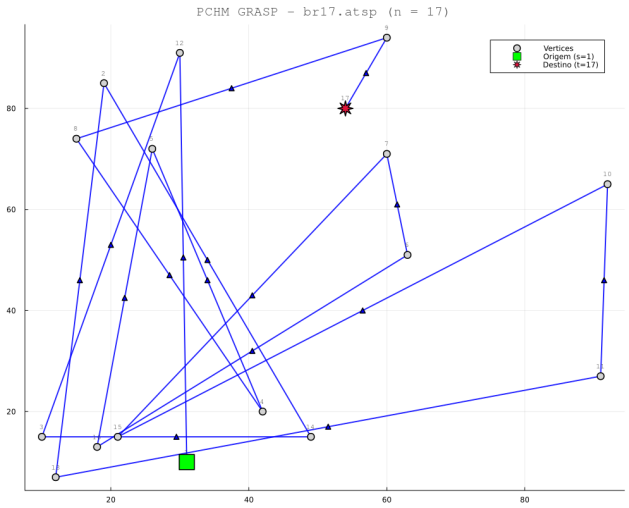


Figura 1: `br17.atsp` ($n = 17$, $Z_{\text{GRASP}} = 34,00$)

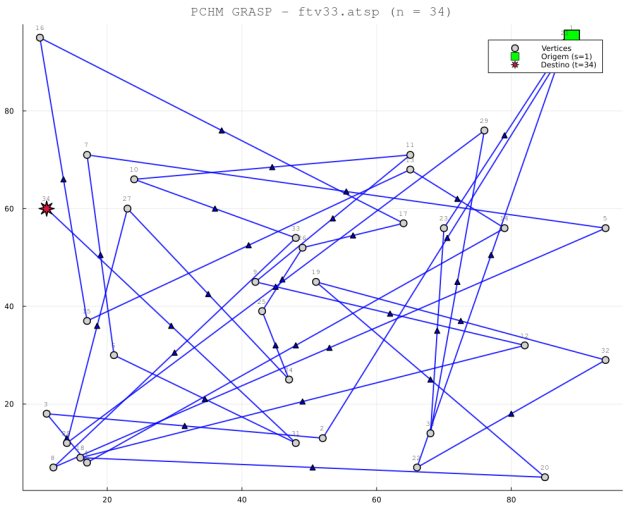


Figura 2: `ftv33.atsp` ($n = 34$, $Z_{\text{GRASP}} = 1275,00$)

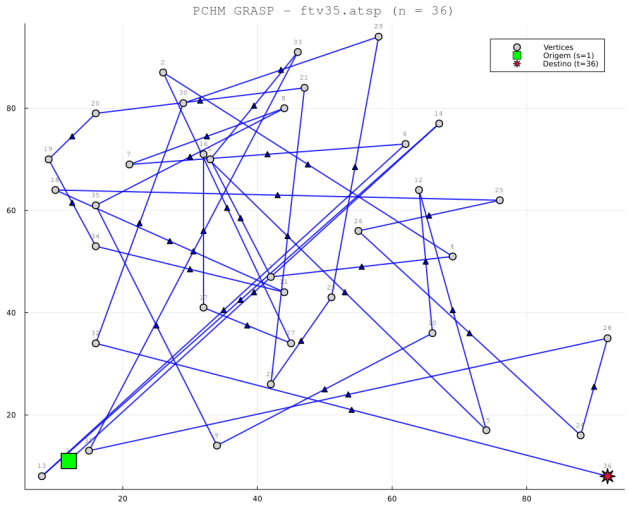


Figura 3: `ftv35.atsp` ($n = 36$, $Z_{\text{GRASP}} = 1455,00$)

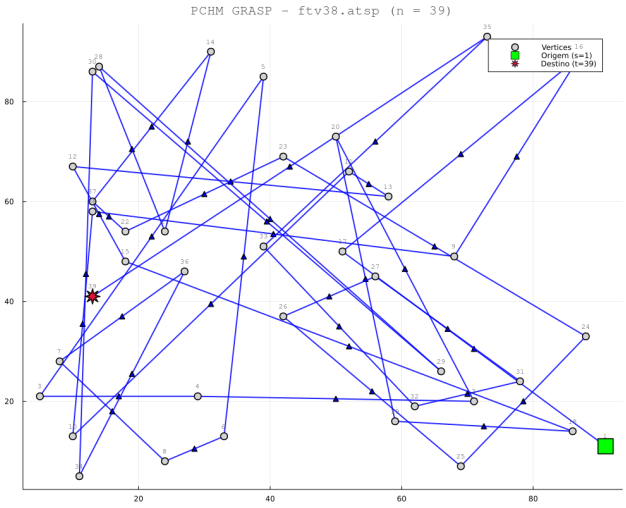


Figura 4: `ftv38.atsp` ($n = 39$, $Z_{\text{GRASP}} = 1520,00$)

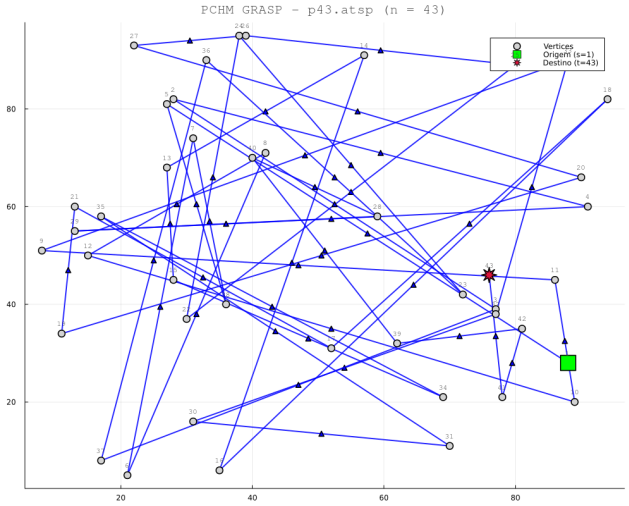


Figura 5: `p43.atsp` ($n = 43$, $Z_{\text{GRASP}} = 596,00$)

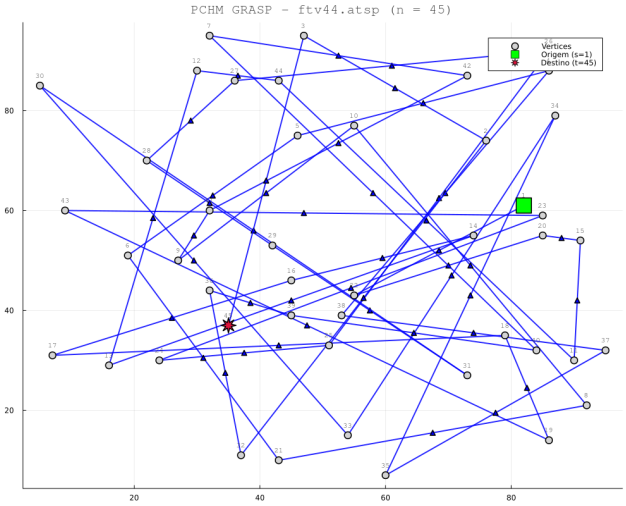


Figura 6: `ftv44.atsp` ($n = 45$, $Z_{\text{GRASP}} = 1654,00$)

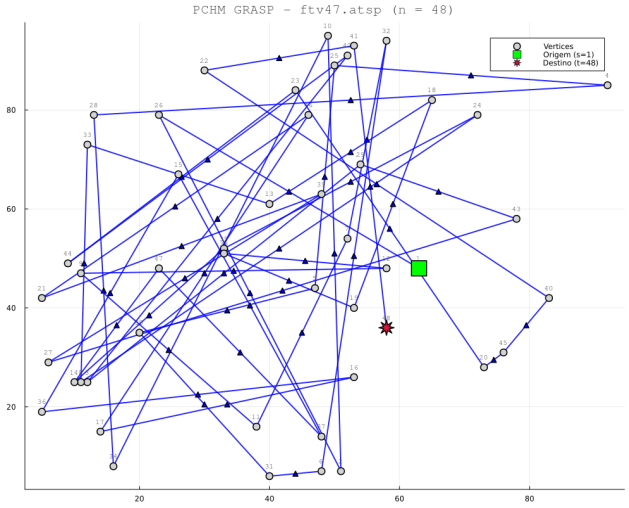


Figura 7: `ftv47.atsp` ($n = 48$, $Z_{\text{GRASP}} = 1796,00$)

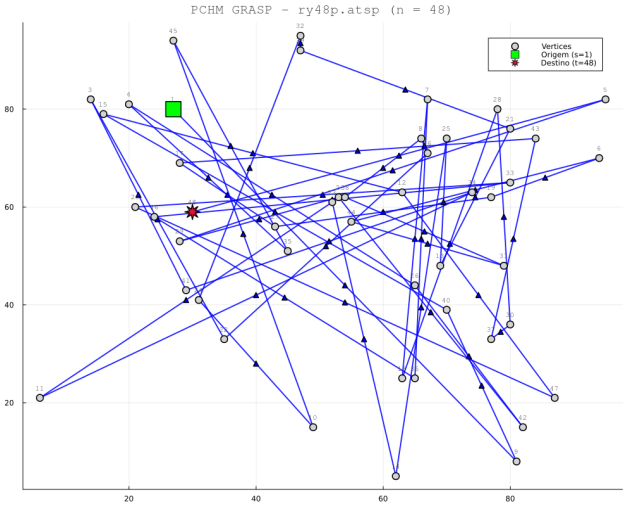


Figura 8: `ry48p.atsp` ($n = 48$, $Z_{\text{GRASP}} = 14231,00$)

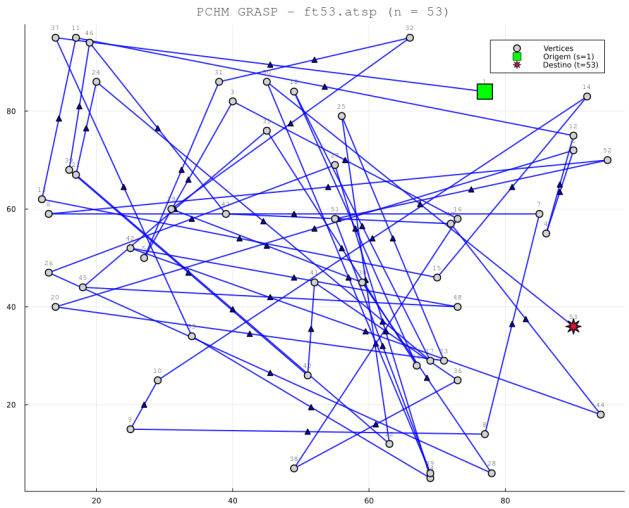


Figura 9: `ft53.atsp` ($n = 53$, $Z_{\text{GRASP}} = 6950,00$)

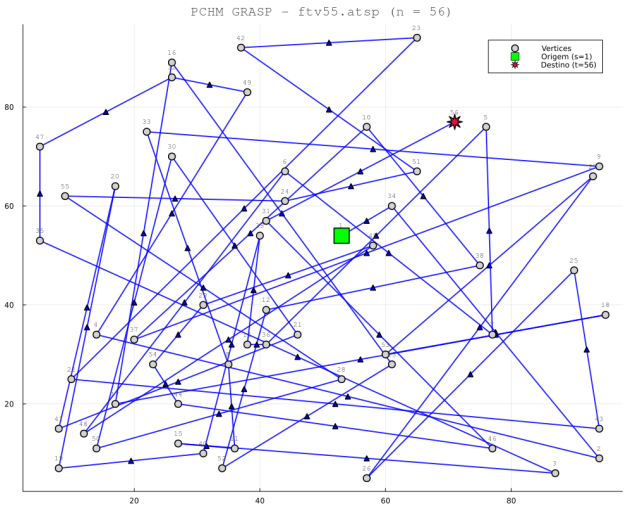


Figura 10: `ftv55.atsp` ($n = 56$, $Z_{\text{GRASP}} = 1696,00$)

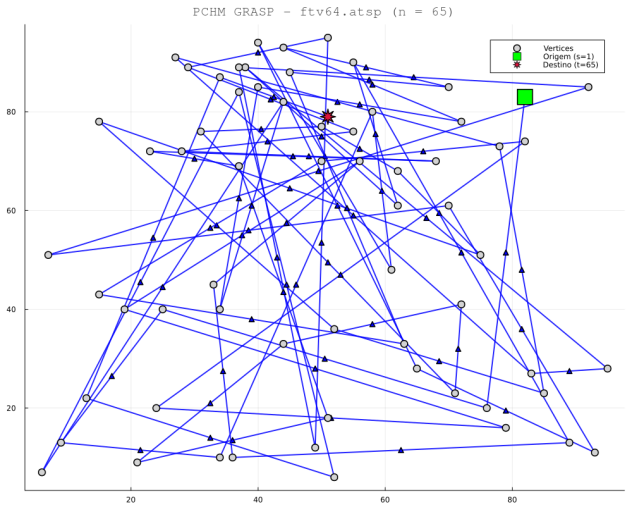


Figura 11: `ftv64.atsp` ($n = 65$, $Z_{\text{GRASP}} = 1975,00$)

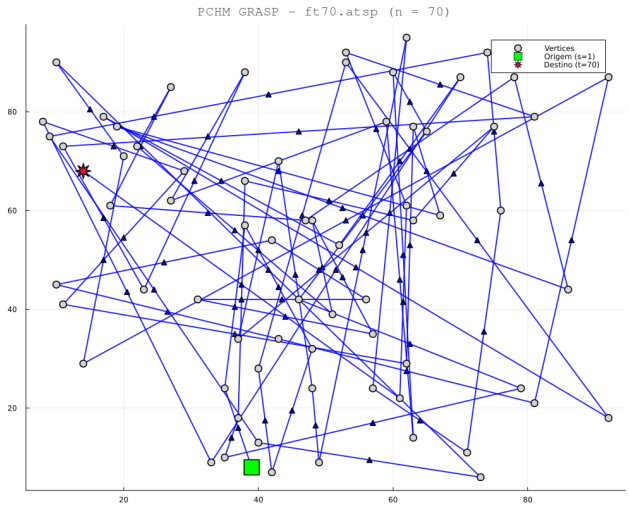


Figura 12: `ft70.atsp` ($n = 70$, $Z_{\text{GRASP}} = 39608,00$)

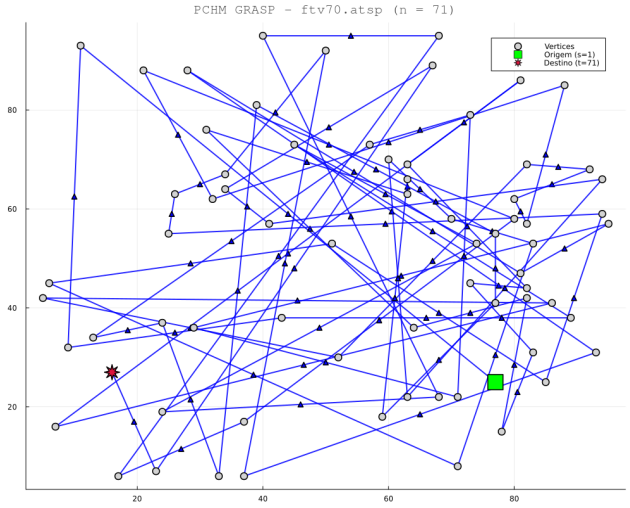


Figura 13: `ftv70.atsp` ($n = 71$, $Z_{\text{GRASP}} = 2104,00$)

6. Declaração de Uso de Inteligência Artificial (Atividade 3.1)

Foi utilizada como apoio na estruturação e revisão do código das heurísticas, consulta sobre as vizinhanças de busca local (2-Opt, Or-Opt e Swap) e formatação deste relatório em LaTeX.